WEEK 06

기능 단위 분리 (함수)

기능 단위 분리

6주차 개요: 기능 단위 분리(함수)

복잡한 작업을 **기능 단위(함수)**로 분리하여 코드의 **재사용성**과 **가독성**을 높이는 방법을 학습합니다. Turtle 그래픽을 도구로 활용하여 함수 정의, 호출, 매개변수 활용법을 실습합니다.

학습 목표

- 복잡한 작업의 기능 단위 분해
- 함수 정의(def)와 호출 실습
- 코드 재사용성과 가독성 향상

실습 도구

- **Turtle 모듈** 기반 그래픽 실습
- 파이썬 함수(Function) 문법
- 매개변수(Parameter) 전달

학습 범위

- 함수 정의/호출, 매개변수, 구조화
- 간식세트 만들기(개념), 집 그리기(실습)
- **주간 과제:** 마을 꾸미기

간식세트 만들기

기능 단위로 분리하는 컴퓨팅 사고

복잡한 전체 작업을 세부적인 기능 단위로 나누어 생각합니다. 각 단계는 독립적인 함수로 구현될 수 있으며, 이를 통해 코드의 구조를 명확히 하고 재사용성을 높일 수 있습니다.

1. 퍼지를 만든다
2. 초코렛 칩 쿠키를 만든다
3. 땅콩 버터 바를 만든다
4. 쟁반에 배치한다

재사용의 필요성: 집 그리기 구성요소

'집 그리기'와 같이 복잡한 그림은 지붕, 몸체, 창문, 문과 같은 기능 단위로 분리할 수 있습니다. 각 부분을 함수로 만들면 코드의 **재사용성(Reuse)**이 높아집니다.

지붕 (Roof)

삼각형 그리기
함수를 호출하여
지붕 생성

몸체 (Body)

사각형 그리기
함수를 재사용하여
벽면 생성

창문/문

작은 사각형을
반복 재사용하여
세부 요소 표현

함수 재사용의 효과

- 코드의 중복을 줄여줍니다.
- 프로그램 수정이 쉬워집니다.
- 전체 구조 파악이 용이합니다.

함수 정의 기본 문법

파이썬에서 함수를 정의하는 규칙과 구조를 이해합니다. 함수는 **def** 키워드로 시작하며, 본문은 **들여쓰기**로 구분됩니다.

함수 정의 규칙

1. **def** 키워드: 함수 정의의 시작을 알립니다.
2. **함수 이름**: 기능을 잘 나타내는 이름을 지정합니다.
3. **콜론(:)**: 헤더의 끝을 알리며, 다음 줄부터 본문이 시작됩니다.
4. **들여쓰기**: 함수에 포함된 코드는 반드시 들여써야 합니다.

def 키워드

함수를 선언할 때 사용하는 예약어입니다.
`def 함수이름():` 형태로 작성합니다.

들여쓰기와 호출

함수 내부 코드는 **Tab** 또는 **공백 4칸**으로 들여씁니다.
정의된 함수는 함수이름()으로 호출하여 실행합니다.

```
# 함수 정의 (Definition)
def hello():
    print("안녕하세요!") # 들여쓰기 필수

# 함수 호출 (Call)
hello()
```

예제: 사각형 그리기 함수

square() 함수 정의와 코드

def 키워드를 사용하여 사각형을 그리는 명령어들을 하나의 기능 단위로 묶어 정의합니다.

함수를 정의한 후에는 **함수이름()** 형태로 호출하여 코드를 재사용할 수 있습니다.

01

1. 모듈 준비 및 객체 생성

```
import turtle
t = turtle.Turtle()
t.shape("turtle")
```

02

2. 함수 정의 (Define)

```
def square():
    t.forward(100)
    t.left(90)
    t.forward(100)
    t.left(90)
    t.forward(100)
    t.left(90)
    t.forward(100)
```

03

3. 함수 호출

```
square()
```

예제: 원 그리기 함수 drawCircle()

반복되는 원 그리기 작업을 함수로 정의하여 재사용성을 높입니다. 펜의 굵기와 색상을 설정하여 다양한 스타일의 원을 표현할 수 있습니다.

구현 포인트

1. **def** 키워드로 drawCircle 함수를 정의합니다.
2. **t.circle(반지름)** 명령어로 원을 그립니다.
3. **pensize()**와 **pencolor()**로 스타일을 지정합니다.

함수 정의

원을 그리는 일련의 명령어들을 하나의 함수 이름 아래 묶어서 관리합니다.

스타일 설정

선의 두께(pensize)와 색상(pencolor)을 변경하여 시각적 효과를 더합니다.

```
import turtle
t = turtle.Turtle()

def drawCircle():
    t.pensize(10) # 펜 굵기 10
    t.pencolor("blue") # 펜 색상 파랑
    t.circle(100) # 반지름 100인 원

drawCircle() # 함수 호출
```

함수 호출과 복합 그림

square()와 drawCircle() 조합

미리 정의해 둔 함수들을 순서대로 호출하면 복잡한 그림도 쉽게 완성할 수 있습니다. 기능 단위로 나누어진 블록들을 조립하여 더 큰 프로그램을 만드는 프로그래밍의 핵심 원리입니다.

01
순차적 호출
(Sequence)

02
기능 재사용
(Reuse)

03
복합 도형
(Composition)

매개변수 개념

함수와 외부 세계를 연결하는 통로

매개변수(Parameter)는 함수가 실행될 때 외부로부터 필요한 값을 전달받는 변수입니다. 이를 통해 고정된 동작이 아닌, 입력값에 따라 유연하게 변화하는 함수를 만들 수 있습니다.

역할 (Role)

함수 외부의 데이터를 내부 로직으로 전달하는 입력 통로 역할을 수행합니다.

장점 (Advantages)

하나의 함수로 크기, 색상, 위치 등을 다르게 처리할 수 있어 코드의 재사용성과 유연성이 극대화됩니다.

규칙 (Rules)

함수 정의 시 소괄호 ( ) 안에 쉼표로 구분하여 나열하며, 호출 시 정의된 순서와 개수를 지켜야 합니다.

예제: 가변 사각형 함수

함수 정의

`def square(width, height):`

가로(width)와 세로(height) 길이를 **매개변수**로 받아 다양한 크기의 사각형을 그릴 수 있도록 정의합니다.

동작 원리

`전진(w) → 회전 → 전진(h)`

가로 길이만큼 전진하고 90도 회전, 세로 길이만큼 전진하고 90도 회전을 두 번 반복하여 직사각형을 완성합니다.

함수 호출

`square(100, 200)`

함수를 호출할 때 괄호 안에 구체적인 값을 전달합니다. 100은 width로, 200은 height로 전달되어 실행됩니다.

```
def square(width, height):
    t.forward(width); t.left(90)
    t.forward(height); t.left(90)
    t.forward(width); t.left(90)
    t.forward(height); t.left(90)
```

사용자 입력 연동: drawCircle(size)

input() 함수와 매개변수의 연결

사용자로부터 입력받은 값을 변수에 저장하고, 이 변수를 함수의 매개변수로 전달하여 동적으로 그림을 그리는 방법을 학습합니다. 외부 데이터가 함수 내부로 흐르는 과정을 이해합니다.

```
a = int(input('사이즈: '))
```

```
def drawCircle(size):
t.circle(size)
```

drawCircle(a) 호출
(변수 a의 값이 size로 전달됨)

다중 매개변수: 원 그리기

형태와 두께

`radius, size`

원의 **반지름(radius)**으로 크기를 결정하고, **선 두께(size)**를 매개변수로 받아 다양한 굵기의 도형을 표현합니다.

색상 스타일

`color`

펜의 **색상(color)**을 문자열(예: "red", "black")로 입력받아 원의 테두리 색을 동적으로 변경합니다.

시작 위치

`x, y`

그림을 그리기 시작할 **좌표(x, y)**를 전달받아, `penup()`과 `goto()`를 이용해 해당 위치로 이동합니다.

```
# 함수 호출 예시: 반지름 100, 검은색, 두께 15, 좌표(0,0)
drawCircle(100, "black", 15, 0, 0)
```

코드 구조화 연습

기능 단위로 묶기 & 리팩터링 실습

제공된 Turtle 그래픽 코드를 분석하고, 눈(), 코(), 입()과 같이 의미 있는 기능 단위로 함수를 만들어 코드를 재구성합니다. 이를 통해 코드의 중복을 제거하고 가독성을 높이는 리팩터링 과정을 학습합니다.

1. 동작 파악: 기존 코드 실행 및 분석
2. 기능 분리: 눈(), 코(), 입() 함수화
3. 리팩터링: 중복 제거 및 구조 개선

주간 과제: 마을 꾸미기

학습한 **함수 정의**와 **매개변수**를 활용하여 나만의 창의적인 마을을 꾸며봅니다. 집, 나무, 꽃 등 최소 4개 이상의 요소를 함수로 만들어 조립해 보세요.

함수 4개 이상

집(), 나무(), 해(), 꽃() 등 최소 4개의 서로 다른 함수를 정의하여 사용

매개변수 활용

좌표(x,y), 색상(color), 크기(size)를 매개변수로 받아 다양성 부여

창의적 구성

반복문이나 랜덤 함수를 활용하여 풍성하고 창의적인 마을 완성

</> 과제 코드 구조 예시

```python
def draw_house(x, y, color):
    t.penup(); t.goto(x, y); t.pendown()
    # 집 그리기 코드...

def draw_tree(x, y):
    # 나무 그리기 코드...

# 메인 실행: 함수들을 조립하여 마을 구성
draw_house(-100, 0, "red")
draw_house(100, 0, "blue")
draw_tree(150, -50)
```

6주차
요약 및 예고

오늘 실습을 통해 복잡한 작업을 기능 단위(함수)로 분리하고, 매개변수를 활용해 코드의 재사용성을 높이는 방법을 학습했습니다. 또한, Turtle 그래픽을 이용해 구조화된 마을 꾸미기 과제를 수행했습니다.

핵심 포인트 & 다음 주차:

01
함수 기초:
정의, 호출, 구조화

02
매개변수:
입력 전달, 일반화

NEXT
알고리즘 표현:
순서도,
Flowgorithm